

FedSGT: Exact Federated Unlearning via Sequential Group-based Training

Bokang Zhang
Emory University
bokang.zhang@emory.edu

Hong Guan
Arizona State University
hguan6@asu.edu

Hong Kyu Lee
Emory University
hong.kyu.lee@emory.edu

Ruixuan Liu
Emory University
ruixuan.liu2@emory.edu

Jia Zou
Arizona State University
Jia.Zou@asu.edu

Li Xiong
Emory University
lxiong@emory.edu

ABSTRACT

Federated Learning (FL) enables collaborative, privacy-preserving model training, but supporting the “Right to be Forgotten” is especially challenging because data influences the model through distributed and interleaved client updates. Existing exact unlearning methods typically require frequent retraining from scratch, resulting in high communication cost and long service downtime. To address this, we propose Federated Sequential Group-based Training (FedSGT), an exact unlearning framework for FL. FedSGT partitions the data into uniform groups, and each client may participate in multiple groups. To control communication overhead, each client can limit the number of groups it contributes to. FedSGT then trains multiple sequences of Parameter-Efficient Fine-Tuning (PEFT) modules, each corresponding to a different group permutation. Since the PEFT modules are lightweight and maintained server-side, FedSGT isolates the influence of different data groups into independent modules without incurring significant storage overhead and communication cost. Exact unlearning is thus achieved instantly by deactivating the modules corresponding to the group containing the unlearned data. Furthermore, using multiple training sequences helps maintain high model utility as deletion requests accumulate. We provide a rigorous theoretical analysis of both the deletion rate—expected number of deletions before retraining is needed—and the expected model performance. Experiments on various tasks demonstrate that FedSGT achieves a significantly longer service maintenance under multiple unlearning requests while maintaining comparable learning performance and training efficiency to other exact unlearning baselines. Extensive ablation studies validate the robustness of our method across a wide range of parameter settings.

1 INTRODUCTION

Federated Learning (FL) [15, 23, 33, 54] has emerged as a dominant privacy-preserving machine learning paradigm, enabling multiple data silos to collaboratively train a global model without sharing their local raw data. This distributed nature has led to its widespread adoption in privacy-sensitive domains such as finance and healthcare. Recently, the increasing stringency of data privacy regulations, such as the GDPR [41], has granted users with the *Right to be Forgotten*, mandating the removal of their data contributions from models. Merely deleting data from a database is insufficient, as model parameters may still retain “memories” of the deleted data. Consequently, Machine Unlearning (MU) [19, 34, 40, 48] techniques

have been developed to efficiently erase the influence of specific data from a trained model. Applying this requirement to FL creates the challenging problem of Federated Unlearning (FU), which aims to remove specific clients’ data from the global model as if it had never been trained on the data.

Existing MU approaches are broadly categorized into approximate unlearning [11, 20, 24], and exact unlearning [2, 4, 49, 51]. Approximate methods modify or fine-tune the model to weaken the impact of the forgotten data, but they typically fail to provide a theoretical guarantee of complete removal, which is unsuitable for applications demanding stringent security and privacy. In contrast, exact unlearning ensures that the unlearned model behaves indistinguishable from one retrained from scratch on the remaining data, providing strong verifiability guarantees.

In the federated setting, achieving exact unlearning remains especially challenging because data influences the model through distributed and interleaved client updates. Most existing FU approaches adopt approximate strategies—such as gradient ascent [35] or partial retraining [25]—but lack rigorous guarantees and often leave residual data traces in the model. Current exact FU methods [36, 39, 50], such as *FedCIO* [36], extend the centralized SISA (Sharded, Isolated, Sliced, and Aggregated) [2] training framework to the distributed case. They partition clients into disjoint clusters and train cluster-specific models independently. Upon an unlearning request, only the cluster containing the deleted data must be retrained. While this approach reduces retraining scope, its reliance on cluster retraining still incurs high communication costs and frequent service downtime, particularly when unlearning requests arrive sequentially.

To overcome the high retraining cost and service failure limitations, we draw inspiration from the recently proposed centralized *Sequence-aware Sharded Sliced Training* (S3T) framework [6], which integrates Parameter Efficient Finetuning (PEFT) to achieve parameter isolation. S3T sequentially trains lightweight PEFT modules with distinct data partitions, enabling exact unlearning by simply deactivating the modules associated with deleted data. Furthermore, S3T trains on multiple permutations (sequences) of the data slices to enhance robustness against multiple deletion requests. Retraining is deferred until all sequence variants for a shard are exhausted, at which point S3T initiates retraining to recover model accuracy.

However, extending S3T to FL introduces challenges due to its distributed nature. First, it is non-trivial to generalize the notion of a “slice” to FL. A natural attempt is to treat a cluster in FedCIO as a slice. However, under Non-IID and uneven client data, it becomes

difficult to form balanced data groups (i.e., clusters) for sequence-aware training across clients, which cannot only degrade accuracy, but also induce larger fluctuations in both the learning [22] and unlearning processes. Second, developing theoretical models to compute the deletion rate (i.e., the expected number of deletions before retraining) and model performance are often required for exact model unlearning, because system designers must predict service availability under long-running deletion workloads and choose partition/sequence budgets without exhaustively tuning over many possible deletion patterns. However, it is particularly challenging in FL environments due to the interaction between grouping, sequencing, and stochastic unlearning.

Contributions. In this paper, we propose FedSGT, an exact federated unlearning framework. We propose to allow clients to partition their data and join multiple groups and dispatch local slices of data to these groups. Since the goal is to partition all clients’ data into uniform groups, a client may participate in multiple groups by contributing local slices of data to each group to balance the data distribution, while its group membership (i.e., local slices) is constrained to keep communication cost under control. We formalize this problem and propose a randomized balanced grouping strategy and a Barycenter-aware centering grouping strategy to resolve the problem. FedSGT then trains multiple sequences of PEFT modules, each corresponding to a different group permutation. This design isolates the influence of different data groups into lightweight, independent modules, enabling instant exact unlearning - the server can simply deactivate modules corresponding to the unlearning data without retraining. Furthermore, using multiple training sequences with our designed sequence selection method, FedSGT maintains high model utility even as deletion requests accumulate. To complement the system design, we also develop a theoretical analysis leveraging probabilistic models of sequential deletions and dependency-aware training structures. It provides principled guidance for parameter selection and demonstrates FedSGT’s service-maintenance advantage and low communication overhead over alternative exact unlearning baselines.

Our primary contributions are summarized as follows:

- We propose FedSGT, a federated exact unlearning framework that efficiently partitions client data into groups mapped to lightweight PEFT modules. Each client contributes to a limited number of groups, achieving exact unlearning through instant module deactivation while keeping the training-phase communication cost bounded.
- We provide a rigorous theoretical analysis of the performance and efficiency for both FedSGT and existing baselines. This includes analyzing their deletion rate, the expected model performance, and the training efficiency (i.e., communication overheads).
- We conduct extensive experiments demonstrating that FedSGT achieves comparable learning performance and training efficiency to baselines, while ensuring significantly longer service maintenance cycles under continuous unlearning requests, where baselines suffer from early service failure. We also conduct extensive ablation studies to validate the robustness of our method.

2 PRELIMINARIES

Machine Unlearning. Machine unlearning is broadly classified into two categories: approximate and exact unlearning. Approximate unlearning [11, 20, 24] aims to efficiently modify a trained model’s parameters to approximate a state as if the target data had never been included. This is often achieved through methods like gradient ascent or by using designated retention and forget sets. However, while efficient, the stochastic nature of this process makes it difficult to audit and may provide weak privacy guarantees.

Exact unlearning [2, 4, 51], in contrast, provides a formal guarantee that the data’s influence is completely removed by modifying the original training process. The seminal work in this area is SISA [2] (Sharded, Isolated, Sliced, and Aggregated training). SISA trains an ensemble of models, each on a disjoint data shard. To reduce costs, each shard is further divided into slices, and the model is trained incrementally, saving checkpoints after each slice. Upon a deletion request, SISA achieves exact unlearning by reverting to the checkpoint before the affected slice and retraining only on the remaining data within that shard. However, SISA’s single sequential model is fragile; deleting an early slice can invalidate all subsequent checkpoints, causing a catastrophic “service failure” that requires a costly full retrain. S3T [6] addresses this issue by training multiple models on different permutations (sequences) of the data slices within a given budget. When a deletion request invalidates one sequence, S3T ensures longer service maintenance by instantly switching to the best-performing model from the remaining, still-valid sequences.

Federated Unlearning. Prior works on federated unlearning (FU) can be broadly categorized as approximate or exact. Approximate FU methods [3, 12, 26, 35, 46, 46] attempt to efficiently estimate the unlearned model. Some approaches, like FedEraser [26], are history-dependent, utilizing stored historical updates to approximate the model state post-deletion. Other methods, such as FUPGA [12, 35], are gradient-based, attempting to reverse a client’s contribution by applying gradient ascent. However, these approximate methods have notable limitations. History-dependent approaches incur impractical storage and communication overhead, as they must save numerous client updates or model checkpoints. Gradient-ascent techniques risk gradient conflicts with remaining data, which can destabilize the global model and degrade its utility, especially in Non-IID settings. Most importantly, neither approach provides a formal guarantee of complete data removal, which is often a strict requirement.

Exact FU methods [36, 39, 50], in contrast, provide a verifiable guarantee that the data’s influence is completely removed in FL. A state-of-the-art example is FedCIO [36], which adapts the SISA framework to the federated setting. FedCIO’s strategy involves partitioning clients into disjoint clusters, training an independent model for each cluster in isolation, and performing a one-shot aggregation to form the global model. While FedCIO achieves exactness, its limitation is its unlearning mechanism: upon a deletion request, it must retrain the entire affected cluster from scratch. This “retrain-from-scratch” paradigm is computationally prohibitive, incurs high latency, and leads to the catastrophic service downtime that our

Table 1: Summary of notations.

Notation	Description
L	Total number of groups in FedSGT
c	Total number of clusters in FedCIO
B	Training budget (number of permutations)
B'	Effective budget, $\min(L, B)$
G_i	The i -th data group in FedSGT
$\mathbb{P}$	Probability distribution of models
N	Clients number
$\mathcal{D}, \mathcal{D} $	Total training data and datasize
$\delta(\cdot)$	Deletion Rate (expected requests until failure)
H_n	n -th Harmonic Number $\sum_{i=1}^n \frac{1}{i}$
r	Number of unlearning requests (deletions)
R	Remaining samples in the best sequence post-unlearning
U	Cyclic Span (min. contiguous groups covering)
T	Communication rounds in FL system
$S(r, m)$	Stirling number of the second kind

work seeks to resolve. In addition, the accuracy and learning stability of FedCIO will degrade with Non-IID data since its grouping of clients by gradient similarity cannot ensure balanced groups.

Federated Parameter-Efficient Fine-Tuning. The ever-increasing size of large-scale models makes direct fine-tuning prohibitively expensive. To mitigate this, Parameter-Efficient Fine-Tuning (PEFT) methods have been proposed. These approaches introduce a small number of additional trainable parameters, while keeping the vast majority of the pre-trained parameters frozen. Among these methods, LoRA [14] is arguably the most popular, achieving performance comparable to full fine-tuning while training less than 1% of the total parameters.

While PEFT addresses the computational cost, the data required for fine-tuning is often domain-specific, siloed across multiple parties, and cannot be shared directly due to privacy concerns. Recent works propose Federated PEFT [18, 55] for reducing communication and computation cost in FL by only training, communicating and aggregating the small set of PEFT parameters (e.g., LoRA matrices) instead of the entire model. We include a broader discussion of related work in Appendix E.

3 PROBLEM STATEMENT AND ANALYSIS

In this section, we formalize the federated learning setup and the exact unlearning problem. We begin by introducing the problem formulation, including both the learning stage and the unlearning stage within a federated system. We then outline the key design goals for an exact federated unlearning framework that preserves model utility, supports long-term service maintenance, and remains training-efficient. Finally, we review existing federated unlearning solutions and highlight their limitations, particularly their inability to support multiple unlearning requests efficiently due to high communication and retraining costs. These limitations motivate the need for a scalable and communication-efficient exact unlearning approach. The main notations used in this paper are summarized in Table 1.

3.1 Problem Statement

Learning Stage. We consider a federated fine-tuning setting where a central server coordinates a set of N participating clients $C = \{c_1, c_2, \dots, c_N\}$. Each client c_i holds a local dataset $\mathcal{D}^i$, which is potentially Non-IID across clients. We focus on the fine-tuning setting where the global model is initialized with a pre-trained backbone (e.g., a large vision or language model) and is collaboratively fine-tuned across clients to adapt to downstream tasks.

To reduce communication and computation costs, only lightweight PEFT modules are trained and exchanged between clients and the server, while the backbone parameters remain frozen and stored locally throughout the process.

Unlearning Stage. Our goal is to achieve **exact unlearning** on the global model, ensuring that the unlearned data is not included in the training data of the updated model. Let $\mathbf{M}$ denote the global model obtained after the learning stage, $\tilde{\mathbf{M}}$ denote the global model after unlearning. Formally, for an unlearning request targeting a forget set $\mathcal{D}_f$, the unlearning operator $\mathcal{U}$ produces $\tilde{\mathbf{M}}$ such that:

$$\mathbb{P}(\mathcal{U}(\mathcal{D}, \mathcal{D}_f, \mathbf{M})) = \mathbb{P}(\mathcal{A}(\mathcal{D} \setminus \mathcal{D}_f)),$$

where $\mathcal{A}$ is the original learning algorithm and $\mathcal{D} \setminus \mathcal{D}_f$ is the remaining dataset.

In an FL system, unlearning requests are triggered by clients—e.g., a client c_i requests that their data $\mathcal{D}^i$ be removed in whole or in part. As such requests may arrive *sequentially*, the server must incrementally update the global model in response to each request, ensuring that the current model does not retain any residual influence from the removed data. Moreover, the system must preserve service availability throughout the unlearning process: the time to service recovery after each deletion must be minimized to avoid extended downtime or performance degradation.

Design Goals. Given the above setting, we aim to develop an exact federated unlearning protocol with the following design goals:

- **G1: Model Utility.** The framework aims for high model utility across the *learning* and the *unlearning* stages. During learning, the global model $\mathbf{M}$ should minimize the expected loss over all client data. During unlearning, the unlearned model $\tilde{\mathbf{M}}$ must maintain comparable utility on the remaining dataset $\mathcal{D} \setminus \mathcal{D}_f$.
- **G2: Service Maintenance.** The framework aims to reduce the service downtime. We should maximize the number of supported unlearning requests before a full retraining becomes necessary. Additionally, the communication or computation cost for each necessary retraining should be small.
- **G3: Training Efficiency.** The training process $\mathcal{A}$ must introduce only modest computational and communication overhead, compared to standard federated learning with PEFT.

3.2 Analysis of Existing Solutions

FedRetrain. The most straightforward baseline is federated retraining (FedRetrain), where for each unlearning request, the server and all clients perform a complete retraining process on the remaining data. While this approach naturally satisfies **G1** (Model Utility) and avoids additional communication overhead, its major drawback lies in extremely high retraining cost and prolonged service downtime. Thus, FedRetrain fails to meet **G2** (Service Maintenance) and

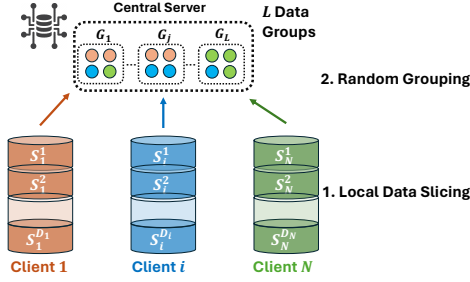


Figure 1: Illustration of the federated data grouping method. Each client locally slices its dataset into multiple pieces, and the central server randomly shuffles and assigns these slices into balanced groups for sequential group-based training.

G3 (Training Efficiency), as it cannot efficiently handle sequential unlearning requests or maintain continuous service availability.

FedCIO. For the FedCIO approach [36], during its unlearning stage, when a deletion request occurs, only the affected cluster needs to be retrained, while other cluster models remain available for inference. Although this design improves **G3** (Training Efficiency), its gradient-based clustering of clients may degrade both training accuracy and stability and compromise **G1** (Model Utility). Moreover, it still suffers from substantial retraining time and fails to satisfy **G2** (Service Maintenance) once multiple clusters are affected. Consequently, FedCIO experiences early service failure and cannot sustain long-term service maintenance without retraining.

4 FEDSGT

We present FedSGT (Federated Sequential Group-based Training), an exact federated unlearning framework designed to meet all three design goals: **G1** (Model Utility), **G2** (Service Maintenance), and **G3** (Training Efficiency). The key insight is to achieve *parameter isolation* through multiple lightweight model sequences, where each sequence corresponds to a distinct permutation of L grouped client datasets. To efficiently realize this idea in a federated setting, FedSGT leverages PEFT modules that are trained sequentially on these data groups.

By maintaining these PEFT modules server-side and training them in a cumulative yet separable manner, FedSGT isolates the contribution of each group into independent parameter subsets. This design enables exact unlearning by simply deactivating the PEFT modules associated with the deleted group—without immediate retraining. Consequently, the framework preserves model utility (**G1**), maintains long-term online service (**G2**), and incurs minimal communication and computation overhead (**G3**).

In the following subsections, we detail the core components of FedSGT: Section 4.1 introduces the data grouping strategy that partitions client data into balanced groups; Section 4.2 describes the sequential group-based training process, which trains PEFT modules along multiple ordered sequences; and Section 4.3 presents the unlearning stage, which performs efficient and exact data removal through module deactivation.

4.1 Data Grouping

We consider the dataset partitioned among N clients, each of which is split into D_i local *slices*. These $M = \sum_{i=1}^N D_i$ slices serve as the atomic training units in FedSGT. The goal of the grouping stage is to organize these heterogeneous client slices into multiple balanced groups that facilitate both efficient learning and exact unlearning.

Data Grouping Objective Formulation. Given all client slices $\{S_{i,j}\}$, our goal is to partition them into L groups $\{G_1, \dots, G_L\}$ such that each group remains representative of the overall data distribution. This is crucial for sustaining utility when groups are progressively removed by unlearning requests, when data is Non-IID. Meanwhile, we enforce balanced group sizes for communication fairness and allow each client to participate in at most B_i groups to bound edge-device workload.

We minimize the deviation between each group’s summary statistics and the global statistics. Concretely, letting $\mu(G_g)$ denote the mean slice embedding within group G_g and $\mu(\mathcal{D})$ the global mean, we consider:

$$\min_{\{G_g\}_{g=1}^L} \sum_{g=1}^L \|\mu(G_g) - \mu(\mathcal{D})\|_2^2 \quad (1)$$

$$\text{s.t. } |G_g| \in \{\lfloor M/L \rfloor, \lceil M/L \rceil\}, \forall g \quad (2)$$

$$\sum_{g=1}^L \mathbf{1}(\exists j : (i, j) \in G_g) \leq B_i, \forall i. \quad (3)$$

This formulation is combinatorial due to the discrete assignment of slices to groups.

Computational Hardness. Problem (1) is NP-hard in general due to its discrete slice-to-group assignment nature. Intuitively, even in a highly restricted setting with two balanced groups ($L = 2$ and equal group sizes), minimizing the deviation between each group mean and the global mean becomes equivalent to finding an equal-cardinality partition whose two subset sums are as close as possible. This is exactly the classic balanced partition problem, which is NP-hard [16]. Adding the per-client participation budget (3) further makes exact optimization impractical at FL scale and motivates lightweight heuristics.

Our Solutions. We therefore adopt efficient heuristics: a randomized balanced partition as the default grouping (Algorithm 1) and a Barycenter-Aware Centering (BAC) as an alternative grouping strategy (detailed in Appendix D).

Randomized Balanced Grouping. Given the hardness above, our default strategy is a purely index-driven randomized balanced partition (Algorithm 1): we permute all slice indices using a random seed and evenly split the permuted list into L groups. This yields (i) nearly equal group sizes; (ii) strong mixing across clients in expectation; and (iii) negligible overhead and no dependence on data semantics or model embeddings, avoiding additional privacy risks. This choice is also supported by classical results in randomized load balancing and scheduling: simple randomization often achieves robust expected balance and can match or outperform more structured deterministic strategies, while avoiding NP-hard optimization at runtime [38].

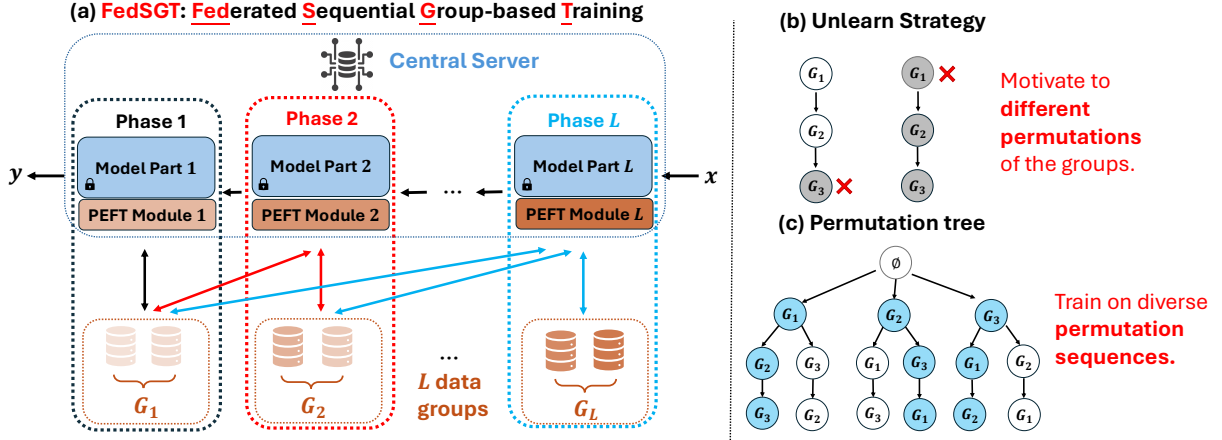


Figure 2: (a) FedSGT overview. Each phase incrementally trains a new PEFT module on the cumulative data groups $\{G_1, \dots, G_i\}$, while freezing previous modules. This design enables exact unlearning by isolating the contribution of each group. (b) Unlearn Strategy: If the earliest trained group (e.g., G_1) is removed, all subsequent modules that depend on it become invalid, motivating the need to diversify training orders. (c) Permutation Tree: To improve service maintenance, multiple group permutations are trained as independent sequences, distributing dependencies across different paths so that unlearning a single group only disables a subset of sequences while keeping others functional.

Barycenter-Aware Centering Grouping and Limitations. We also consider an embedding-aware variant, Barycenter-Aware Centering (BAC), which greedily assigns slices to reduce $\|\mu(G_g) - \mu(\mathcal{D})\|_2$ more directly. At a high level, BAC repeatedly selects a group with the largest barycenter deviation and inserts a slice that best decreases this deviation, under balance and budget constraints. While BAC is more optimization-driven, it has notable limitations in our setting: it introduces extra computation and coordination, and may depend on model states or learned representations, raising additional privacy and systems concerns. Empirically, we find that the simpler randomized strategy already provides sufficiently good representativeness and robustness, often outperforming BAC once we account for runtime overhead and the inherent stochasticity of FL training. Detailed comparisons are deferred to Appendix D.

REMARK 1. Different from typical machine unlearning frameworks such as SISA [2], the definitions of group and slice in our method are distinct. In SISA, the dataset is first divided into multiple shards, each of which is further partitioned into slices trained sequentially. In contrast, our framework considers only a single global shard, while each group corresponds to an independent subset of data—analogueous to a slice in SISA.

4.2 Sequential Group-based Training (SGT)

After grouping, the federated training proceeds in a cumulative and sequential manner, as illustrated in Figure 2. Let $\{G_1, G_2, \dots, G_L\}$ denote the L grouped datasets. The central server maintains a shared backbone model and a series of PEFT modules $\{P_1, \dots, P_L\}$, each corresponding to one training phase.

At Phase 1, the model is trained with the data from G_1 , producing the first PEFT module P_1 . At Phase 2, training continues on the cumulative dataset $\{G_1, G_2\}$, initializing a new PEFT module P_2 while keeping P_1 frozen. This process repeats sequentially—at

Algorithm 1: Data Grouping of Client Slices

1 **Input:** Client set $C = \{c_1, \dots, c_N\}$; per-client slices $\{S_{i,1}, \dots, S_{i,D_i}\}$; number of groups L ; random seed s

2 **Output:** Data groups $\{G_1, \dots, G_L\}$ and mapping $(i, j) \mapsto \text{group_id}(S_{i,j})$

3 **Procedures:**

- 1: $M \leftarrow \sum_{i=1}^N D_i$ // total number of slices
- 2: $\mathcal{I} \leftarrow \{(i, j) \mid i \in [1..N], j \in [1..D_i]\}$ // index pairs for all slices
- 3: $\pi \leftarrow \text{Permute}(\mathcal{I}; s)$ // shuffle indices using random seed s
- 4: $q \leftarrow M/L$ // number of slices per group
- 5: **for** $g = 1$ **to** L **do**
- 6: $G_g \leftarrow \{\pi[(g-1)q+1], \pi[(g-1)q+2], \dots, \pi[gq]\}$ // assign q slices to group g
- 7: **end for**
- 8: **for each** $(i, j) \in \mathcal{I}$ **do**
- 9: set $\text{group_id}(S_{i,j}) \leftarrow g$ **iff** $(i, j) \in G_g$
- 10: **end for**

Return: Group set $G = \{G_1, \dots, G_L\}$ and mapping $(i, j) \mapsto \text{group_id}(S_{i,j})$

Phase i , the model is fine-tuned on $\{G_1, \dots, G_i\}$, generating the PEFT module P_i . Hence, each module P_i encodes the incremental contribution of the i -th group beyond what was already captured by the preceding ones.

It is worth noting that each group G_i may involve data slices from multiple clients. During the training of group G_i , the corresponding clients participate in a standard federated fine-tuning process where each client locally updates the current PEFT module on its local portion of G_i , and the central server aggregates their updates to obtain the global module parameters. Therefore, within

Algorithm 2: Sequential Group-based Training

```

1 Input: Grouped datasets  $\{G_1, \dots, G_L\}$ ; PEFT backbone
   model  $M$ ; training budget  $B$ 
2 Output: Trained model set  $\mathcal{F} = \{M_\rho \mid \rho \in \Pi\}$ 
3 Procedures:
   1:  $\mathcal{F} \leftarrow \emptyset$  // initialize trained model set
   2:  $\Pi \leftarrow \text{SELECTTRAININGSEQUENCES}(L, B)$ 
     // select up to  $B$  group permutations  $\rho$ 
   3: for  $\rho \in \Pi$  do
   4:   Initialize backbone model  $M_\rho \leftarrow M$ 
   5:   for  $i = 1$  to  $L$  do
   6:      $M_\rho \leftarrow \text{GROUPWISETRAINING}(M_\rho, \{G_{\rho(1)}, \dots, G_{\rho(i)}\})$ 
   7:     Freeze PEFT module  $P_{\rho(i)}$ 
     // preserve previous knowledge
   8:   end for
   9:    $\mathcal{F} \leftarrow \mathcal{F} \cup \{M_\rho\}$  // store trained sequence model
10: end for
Return: Trained model set  $\mathcal{F} = \{M_\rho \mid \rho \in \Pi\}$ 

```

each phase, the procedure is fully compatible with conventional federated training, while across phases, it progresses cumulatively through the sequence of groups.

This design achieves two key properties: (1) it enables exact unlearning—removing a group G_i can be done by deactivating all PEFT modules $\{P_i, \dots, P_L\}$ that depend on it; and (2) it ensures training efficiency, as each phase updates only a small number of PEFT parameters while preserving previously learned knowledge. The complete procedure is summarized in Algorithm 2.

Training with Multiple Sequences. While the sequential group-based training described above enables exact unlearning, it still suffers from a key limitation: if the earliest trained group (e.g., G_1) is removed, all subsequent PEFT modules depend on it, and thus the entire model becomes invalid. As illustrated in Figure 2(b), this dependency chain causes early service failure when unlearning requests affect the prefix of the sequence.

To mitigate this issue, we extend the training process to multiple **permutation sequences** of group orders, as shown in Figure 2(c). Instead of relying on a single order $\{G_1, G_2, \dots, G_L\}$, the server trains multiple independent sequential models, each following a distinct permutation of the groups. Each sequence defines an alternative dependency path, enabling the system to recover valid model variants even after certain groups are deleted.

This multi-sequence design substantially improves **service maintenance** by increasing the number of supported unlearning requests before retraining becomes necessary. Training on diverse group permutations distributes dependency structures across multiple paths, ensuring that the removal of any individual group disables only a subset of sequences while keeping others fully functional.

However, introducing more training sequences inevitably increases computational and communication overhead. To balance service robustness and efficiency, we define a training budget B that limits the maximum number of sequences trained. Within this budget, the server selects a subset of representative group permutations

for sequential training to achieve optimal coverage of unlearning scenarios.

4.3 Unlearning

After training, each sequence model M_ρ corresponds to a distinct dependency path among group datasets $\{G_1, \dots, G_L\}$. When an unlearning request arrives for a specific group G_u , the system performs exact unlearning by **deactivating** all PEFT modules that depend on this group. Concretely, for any sequence $\rho \in \Pi$, if G_u appears at position k in ρ , then all subsequent modules $\{P_{\rho(k)}, P_{\rho(k+1)}, \dots, P_{\rho(L)}\}$ in that sequence are disabled, yielding a truncated yet valid model $M_\rho^{(-u)}$. We denote by $M_\rho^{(-u)}$ the truncated model obtained from M_ρ after deactivating all PEFT modules that depend on group G_u . This mechanism ensures that the remaining model contains no contribution from the deleted group, achieving exact and certifiable unlearning without retraining.

However, after certain groups are removed, different sequence models may remain partially functional. Since each sequence covers a distinct subset of the dependency paths, the system must determine which available model to use for inference. This introduces the challenge of sequence selection: given a deletion set $\mathcal{U}$, we must select from the remaining available sequence models $\{M_\rho^{(-\mathcal{U})}\}$ to ensure the best utility and stability during inference.

Sequence Selection Method. After certain groups are unlearned, different sequence models retain different numbers of active PEFT modules. Hence, not all remaining models contribute equally to inference. To ensure stable prediction quality, we introduce a sequence selection mechanism that determines which available model(s) should be used for service and how to combine them. We design three representative strategies:

- **AllSeq:** Aggregate predictions from all available sequences through weighted averaging. The weight of each sequence is proportional to the number of its currently active modules, reflecting the extent of learned knowledge still preserved after unlearning.
- **MinSeq:** Select the sequence whose active modules share the least overlap with the deleted groups across all trained permutations. When multiple sequences remain partially valid, this strategy favors the one most independent from the removed data, thereby minimizing potential interference.
- **LongSeq:** Select the single sequence with the longest active prefix (i.e., the largest number of remaining modules). This strategy prioritizes the sequence with the deepest training depth, which typically preserves the best utility.

This weighted treatment naturally reflects the varying reliability among the remaining sequences: those retaining more activated modules contribute more to the final inference, while truncated sequences have a weaker influence. In Section 5.2, we provide a detailed analysis of the sequence selection strategies and their impact on overall model utility.

EXAMPLE 1. We illustrate how different sequence selection strategies behave under consecutive unlearning requests. Consider the FedSGT system consists of $G = 6$ data groups and $B = 6$ trained sequences: [012345], [501234], [450123], [345012], [234501], [123450]. We simulate three unlearning stages, where the requests sequentially target group data $\{G_1\}$, $\{G_5\}$, and $\{G_2\}$. The resulting sequence and

Table 2: Sequence and model selection results under different sequence selection strategies at three cumulative unlearning stages. The system is configured with $G = 6$ groups and a training budget of $B = 6$. Red highlights (...) indicate the remaining active PEFT modules after each unlearning.

Strategy	Time 1 (unlearning on $\{G_1\}$)	Time 2 (unlearning on $\{G_5\}$)	Time 3 (unlearning on $\{G_2\}$)
AllSeq	[012345], [501234], [450123], [345012], [234501]	[012345], [450123], [345012], [234501]	[012345], [450123], [345012]
MinSeq	[234501]	[012345], [234501]	[012345], [345012]
LongSeq	[234501]	[234501]	[345012]

model selections are summarized in Table 2. In general, the available sequences and models under MinSeq form a subset of those under AllSeq, while LongSeq further selects the longest surviving one from the MinSeq candidates.

5 THEORETICAL ANALYSIS

In this section, we present a theoretical analysis of FedSGT and the baselines (FedCIO, FedAvg). We begin by formalizing the deletion rate for exact unlearning systems, then analyze utility during the unlearning stage and the associated overhead. Building on these theoretical results, we provide a direct comparison between FedSGT and FedCIO to clarify their relative efficiency-utility trade-offs.

5.1 Deletion Rate

DEFINITION 1 (DELETION RATE [6]). The deletion rate, $\delta(S)$, of an exact unlearning system S , is the expected number of deletion requests the system can handle before a full retraining becomes necessary, i.e., when no unaffected model remains available for service.

The deletion rate quantifies a system’s tolerance to unlearning requests. The theoretical results are presented in the main text, with detailed proofs provided in Section A.1.

FedSGT. In FedSGT, when the budget $B > 1$, the system exhibits greater resilience to unlearning requests. Given L total number of groups and a training budget B , we have $B' = \min(L, B)$ different groups at the topmost position in a sequence. The deletion rate quantifies the number of unlearning requests that can be processed before all B' groups are affected, and is given by

$$\delta(\text{FedSGT}) \approx L \cdot H_{B'}.$$

where $H_{B'}$ is the B' -th harmonic number.

FedCIO. In FedCIO, complete failure occurs when all clusters are affected. Assuming all clusters are of equal size and each unlearning request randomly targets one cluster, the process reduces to a coupon collection problem [1].

Let c denote the number of clusters; the expected deletion rate for FedCIO is

$$\delta(\text{FedCIO}) \approx c \cdot H_c,$$

where H_c is the c -th harmonic number.

Deletion rate grows with system degrees: FedSGT increases with the budget B (via $B' = \min(L, B)$) and group count L , while FedCIO increases with the number of clusters c ; both follow harmonic growth.

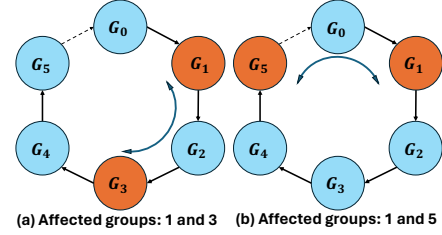


Figure 3: Counting cyclic span. (a) If the affected groups are 1 and 3, the cyclic span is 3 because the minimum clusters to be removed are group 1, 2, and 3; (b) If the affected groups are 1 and 5, the cyclic span is also 3 because the minimum groups to be removed are 5, 0, and 1.

5.2 Performance Guarantee

While direct quantification of model performance without evaluation is challenging, we employ the expected number of remaining training samples R associated with remaining PEFT modules after r unlearning requests as a proxy for performance, given their positive correlation.

FedSGT. Due to space constraints, we present only the analytical results in the main text. Detailed analyses and derivations are provided in Section A.2.

FedSGT employs a cyclic rotation algorithm to create $B > 1$ sequences for each shard. It is difficult to get $\mathbb{E}_{B < L}[R]$ because one needs to analyze many different situations separately. Here we show how to get $\mathbb{E}_{B=L}[R]$, which is also our default setting in our experiments. Since the sequences when $B > L$ include every sequence when $B = L$ when using the cyclic rotation algorithm, so $\mathbb{E}_{B > L}[R] > \mathbb{E}_{B=L}[R]$.

Since FedSGT involves B sequences coupled with different serving strategies (i.e., sequence selection strategies), we cannot simply use the total remaining training samples of all sequences to estimate the model performance. Thereby, we analyze the case for the LongSeq strategy, computing the expectation of the greatest number of remaining training samples in one sequence. It is given by $\mathbb{E}[R] = \frac{|D|}{L}(L - \mathbb{E}[U])$, where U is the *cyclic span*, defined as the minimum number of slices to be removed in a sequence using the cyclic rotation algorithm with $B \geq L$. Figure 3 illustrates two cases of counting cyclic span. The expression for $\mathbb{E}[U]$ is

$$\mathbb{E}[U] = \sum_{m=1}^{\min(r, L)} \Pr(M = m) \cdot \mathbb{E}[U \mid M = m].$$

The first term in the summation is given by

$$\Pr(M = m) = \binom{L}{m} \cdot m! \cdot S(r, m) / L^r$$

where $S(r, m)$ is the Stirling number of the second kind. The second term in the summation is given by

$$\mathbb{E}[U \mid M = m] = 1 + \sum_{s=1}^{L-1} \frac{1}{\binom{L-1}{m-1}} \times \sum_{j=0}^{\lfloor \frac{L-m}{s} \rfloor} (-1)^j \binom{m}{j} \binom{L-1-j}{m-1}$$

FedCIO. Let X_i be the indicator that group i is not affected after r independent uniformly distributed unlearning requests. $P(X_i = 1) = (1 - \frac{1}{c})^r$, so by linearity of expectation, the expectation of number of groups that are unaffected after r unlearning requests is $\mathbb{E}[\sum_{i=1}^c X_i] = \sum_{i=1}^c \mathbb{E}[X_i] = c(1 - \frac{1}{c})^r$. The cluster size is $|\mathcal{D}|/c$, therefore, the size of the training samples that remain is $\mathbb{E}[R] = \frac{|\mathcal{D}|}{c} \cdot c(1 - \frac{1}{c})^r = |\mathcal{D}|(1 - \frac{1}{c})^r$.

5.3 Overhead Analysis

Assumptions. Clients have equal-sized datasets; we set the number of data groups equal to the number of PEFT modules L . We compare FedSGT with FedAvg and FedCIO in training and communication. With the backbone frozen, cost comes from (i) training PEFT modules and (ii) transmitting their updates.

5.3.1 Training Complexity. Let N be the number of clients, E local epochs, T rounds, $|\mathcal{D}| = \sum_{i=1}^N |\mathcal{D}_i|$ the total data size, and P the trainable parameters per PEFT adapter. Updating one module on $|\mathcal{D}_i|$ costs $O(|\mathcal{D}_i|P)$.

FedAvg. All clients fine-tunes a single PEFT module for E epochs over T rounds; across L modules the total training cost is

$$O(T \cdot E \cdot |\mathcal{D}| \cdot P \cdot L).$$

FedCIO. FedCIO partitions the N clients into C disjoint clusters and runs federated fine-tuning within each cluster for T rounds and E local epochs. Let $|\mathcal{D}_k|$ be the data size of cluster k with $\sum_{k=1}^C |\mathcal{D}_k| = |\mathcal{D}|$. The per-cluster cost is $O(TE|\mathcal{D}_k|P)$; summing over clusters (and L modules) yields

$$O\left(\sum_{k=1}^C TE|\mathcal{D}_k|P \cdot L\right) = O(TE|\mathcal{D}|PL),$$

matching FedAvg in overall compute.

FedSGT. Unlike the baselines, FedSGT trains sequential group-based modules: at phase i , a new PEFT module $\mathbf{P}_i$ is trained on $\{G_1, \dots, G_i\}$ while previous modules are frozen. Each phase uses E local epochs and one communication round; only one module is updated at a time, so the total number of rounds does not grow with L .

Let $|G_i|$ be the size of group G_i with $\sum_{i=1}^L |G_i| = |\mathcal{D}|$. The per-sequence cost sums L incremental stages:

$$C_{\text{seq}} = O\left(\sum_{i=1}^L E\left(\sum_{j=1}^i |G_j|\right)P\right) \approx O\left(E \frac{L+1}{2} |\mathcal{D}| P\right),$$

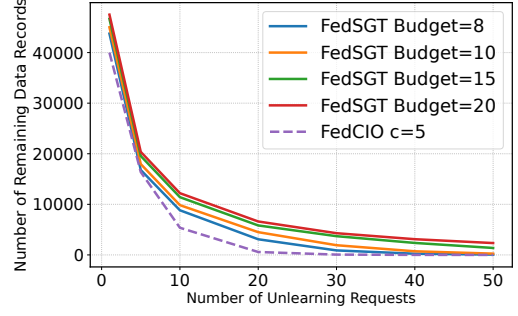


Figure 4: Simulation results of performance analysis

assuming balanced groups $|G_i| \approx |\mathcal{D}|/L$. With a budget of B sequences, the total training cost scales linearly:

$$C_{\text{train}} = B C_{\text{seq}} = O\left(BE \frac{L+1}{2} |\mathcal{D}| P\right).$$

Thus, FedSGT adds linear computation in B without extra communication rounds, trading modest overhead for stronger unlearning robustness and service continuity.

Discussion. In FedSGT, each phase uses a single communication round while keeping the same local epochs as the baselines ($E' = E$). To match the total training cost of FedAvg/FedCIO, equate C_{train} :

$$B \cdot E \cdot \frac{L+1}{2} \approx T \cdot E \cdot L \implies B \approx \frac{2TL}{L+1}$$

5.3.2 Communication Overhead. We evaluate communication overhead from the perspective of how many communication rounds each client participates in on average. This metric better reflects the true client-side cost, since different methods involve clients in different subsets of rounds.

FedAvg. In FedAvg, every client participates in every global communication round. Thus, each client takes part in exactly

$$C_{\text{FedAvg}}^{\text{client}} = T$$

rounds, where T is the total number of FL rounds.

FedCIO. FedCIO adds an initial clustering stage requiring T_{cluster} rounds, where all clients must participate to compute similarity statistics. After clustering, each cluster trains independently for T rounds, but clients still participate in all communication rounds within their own cluster.

Since every client belongs to exactly one cluster, the per-client communication cost becomes

$$C_{\text{FedCIO}}^{\text{client}} = T_{\text{cluster}} + T.$$

FedSGT. In FedSGT, a client's communication cost depends on how many groups it participates in during the L cyclic training sequences. Let K denote the number of groups that contain at least one of its S slices after applying the uniform random grouping in Algorithm 1. Conditioned on $K = k$, we show in Section A.3 that the client's total communication cost is

$$\mathbb{E}[C_{\text{total}}(L) \mid K = k] = L(L+1) \frac{k}{k+1}.$$

Under random grouping, K is itself a random variable whose distribution depends on S . Combining these two components yields the

exact expected communication cost:

$$\mathbb{E}[C_{\text{total}}(L, S)] = L(L + 1) \sum_{k=1}^{\min\{S, L\}} \Pr(K = k) \frac{k}{k + 1}, \quad (4)$$

where $\Pr(K = k)$ is characterized in Section A.3. Equation (4) provides a closed-form expression for communication overhead as a function of the slice granularity S .

Considering the heterogeneity of edge devices in federated learning [9], we further analyze how the slicing granularity affects the performance–overhead trade-off, with detailed results provided in Appendix B.

5.4 Comparison with Baselines

We compare FedSGT with FedCIO under matched training overhead. Consider a 10-client FL system with total data size 50,000, where FedSGT uses $L = 10$ groups and 2 slices per client, and FedCIO uses $c = 5$ clusters.

Matching training cost. From Section 5.3.1, if FedCIO uses $T = 10$ training rounds (plus $T_{\text{cluster}} = 2$ clustering rounds), the corresponding budget for FedSGT is $B \approx 36$. In our comparison, we evaluate smaller budgets $B \in \{8, 10, 15, 20\}$ to study cost–performance trade-offs.

Unlearning utility and deletion rate. Using the theoretical results (Section 5.2), Figure 4 illustrates the expected utility under sequential unlearning requests. As shown, FedSGT consistently outperforms FedCIO across nearly all requests. This aligns with the theoretical deletion rates in Section 5.1: $\delta(\text{FedSGT}) \approx 29.29$ versus $\delta(\text{FedCIO}) \approx 11.42$, meaning FedSGT is expected to maintain service for over $2.5\times$ more deletions.

Training and communication overhead. Under the setting, FedCIO performs 12 communication rounds, whereas FedSGT executes 71.5 rounds (for $B = 10, S = 2$). Although this is more rounds ($\sim 5.9\times$), each FedCIO round transmits all L PEFT modules, while FedSGT transmits only the active group’s modules (often just one or two). Consequently, the total communication bandwidth of the two methods remains comparable.

FedCIO Cluster Numbers. Although FedCIO’s deletion rate improves with more clusters, large c values severely degrade model utility [36], especially under Non-IID data, where training collapses toward per-client learning. We validate this effect experimentally in the next section. Thus, we adopt $c = 5$ here as a practical balance between utility and cluster isolation.

Summary. Under approximately matched training and communication overhead, FedSGT provides strictly higher unlearning utility and a substantially better deletion rate than FedCIO. The next section empirically validates these theoretical findings.

6 EXPERIMENTS

In this section, we conduct extensive experiments to demonstrate and analyze FedSGT’s performance, compared to representative baselines.

6.1 Experiment Setup

Competitors. We compare FedSGT with two representative exact unlearning baselines:

- **FedRetrain.** The most straightforward exact unlearning baseline that fully retrains the federated model from scratch after each unlearning request. While it guarantees correctness, it incurs the highest computation and communication cost.
- **FedCIO.** A state-of-the-art exact FU that extends the centralized SISA framework by dividing clients into disjoint clusters and training each cluster independently.

Datasets and Models. We evaluate FedSGT on both vision and language model tasks to comprehensively assess its generality. For vision benchmarks, we use CIFAR-10 and CIFAR-100 [17], adopting a ViT_{BASE} [7] backbone for federated fine-tuning. For natural language understanding (NLU) tasks, we employ four datasets from the GLUE [42] benchmark—SST-2, QQP, QNLI, and MNLI—using RoBERT_{LARGE} [31] as the base model.

Implementation Details. The experimental setup adopts the following hyperparameter configuration. We consider a federated system with 10 clients, for FedCIO, the number of clusters c is set to 5. The local training epochs are set to 3 and 5 for vision and language tasks, respectively. Both FedAvg and FedCIO are trained with 10 communication rounds, while FedSGT adopts a default training budget of $B = 10$, and local slice data numbers $S = 5$ and AllSeq unlearning sequence selection strategy. As analyzed in Section 5.3, this configuration corresponds to a lower overall training cost compared to the baselines. Both IID and Non-IID settings are evaluated: the Non-IID distribution is simulated using a Dirichlet allocation with concentration parameter $\alpha = 0.3$. For the PEFT backbone, we use LoRA [14] by default due to its broad applicability, efficiency, and ease of deployment across different model architectures. We also evaluate the impact of other PEFT methods on FedSGT, and provide the detailed results in Appendix C. We provide the details of the experimental setup and hyperparameters in Appendix G. The experiments were performed on a server equipped with NVIDIA H200 GPUs and 1 TB of system memory.

6.2 Training Performance

We first evaluate the training performance of FedSGT compared with the two baselines, FedAvg and FedCIO, under both vision and language tasks. Figure 5(a) reports the final test accuracy across all datasets, including CIFAR-10 and CIFAR-100 for vision, and four GLUE benchmarks (SST-2, QQP, QNLI, and MNLI) for natural language understanding.

Overall, FedSGT achieves comparable training performance than FedCIO and remains close to the standard FedAvg baseline, despite introducing the sequential group-based training mechanism. On CIFAR-10, FedSGT almost matches the full fine-tuning accuracy of FedAvg, while maintaining a similar level of stability. On QQP task, FedSGT achieves even slightly better performance than FedAvg and FedCIO, indicating that the introduction of lightweight PEFT modules and sequential updates does not compromise the model’s representation capacity. These results confirm that FedSGT retains the efficiency and effectiveness of standard federated fine-tuning.

Non-IID Setting. We further evaluate all methods under Non-IID client data distributions to investigate the impact of data heterogeneity on training performance. As shown in Figure 5(b), FedSGT consistently achieves higher accuracy than both FedAvg and FedCIO across most tasks. This improvement stems from its uniform

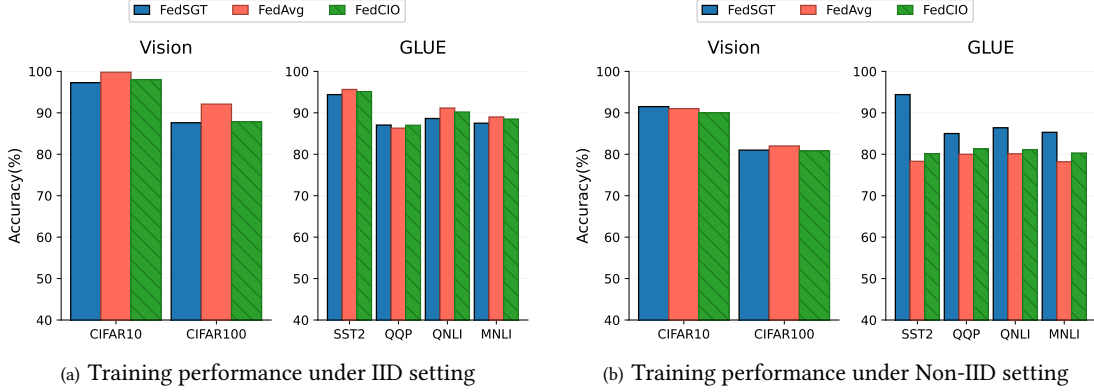


Figure 5: Training performance comparison between FedSGT, FedAvg, and FedCIO under IID and Non-IID settings.

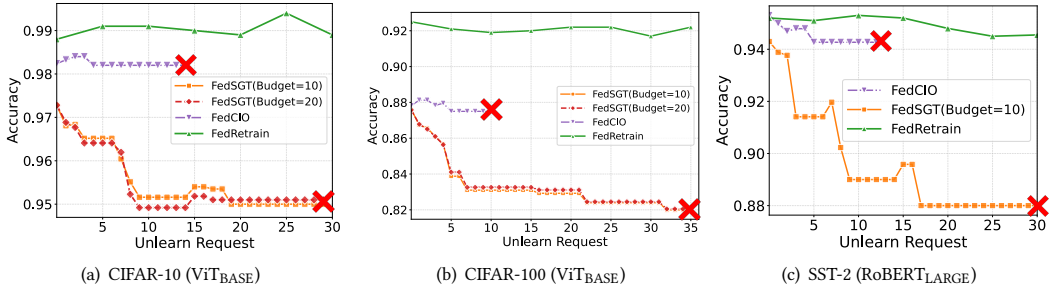


Figure 6: Unlearning Performance on IID setting.

grouping strategy Section 4.1, which balances data representation across clients during sequential training. By forming more homogeneous groups and mitigating the bias introduced by skewed client distributions, FedSGT effectively stabilizes the training process and alleviates performance degradation caused by Non-IID data. In contrast, FedCIO’s disjoint clustering amplifies inter-cluster heterogeneity, leading to less consistent optimization and inferior performance under Non-IID data.

Takeaway 1: FedSGT delivers comparable or even superior training performance to FedAvg and FedCIO under IID settings. Under Non-IID conditions, the uniform grouping strategy of FedSGT effectively mitigates data heterogeneity, leading to more stable and better model performance.

6.3 Unlearning Performance

Unlearning Request Setting. We assume that unlearning requests arrive independently, without any correlation among the deleted data samples. Each request targets a batch of 100 data records contained within a single client slice.

Figure 6 reports the unlearning performance under IID data distributions. For clarity, we evaluate *FedRetrain* every five unlearning requests due to its high retraining cost. Across all datasets, FedSGT maintains service availability for substantially more deletion requests than FedCIO, which aligns with our theoretical deletion rate

analysis in Section 5.1. During early unlearning stages, FedCIO preserves slightly higher utility, as each cluster in the IID setting receives a nearly equal share of the global distribution.

Non-IID Setting. We further evaluate all methods under Non-IID setting to examine the impact of data heterogeneity on unlearning behavior. As shown in Figure 7, FedSGT consistently achieves higher model utility than FedCIO. FedCIO suffers from highly uneven cluster quality in the Non-IID regime: different clusters capture very different data distributions, causing their retrained or retained models to vary dramatically in accuracy. In contrast, FedSGT remains stable because the proposed uniform random grouping strategy (Section 4.1) distributes client slices across groups in a balanced manner, effectively mitigating the effects of Non-IID data during both training and unlearning.

Takeaway 2: FedSGT provides consistently longer service maintenance than FedCIO across both IID and Non-IID settings, supporting substantially more exact unlearning requests before failure. Moreover, under Non-IID distributions, FedSGT achieves markedly higher model utility due to its uniform random grouping, which mitigates the adverse effects of data heterogeneity.

6.4 Training Efficiency

Table 3 reports the end-to-end training time of all methods across vision and language benchmarks. Overall, the total training cost of

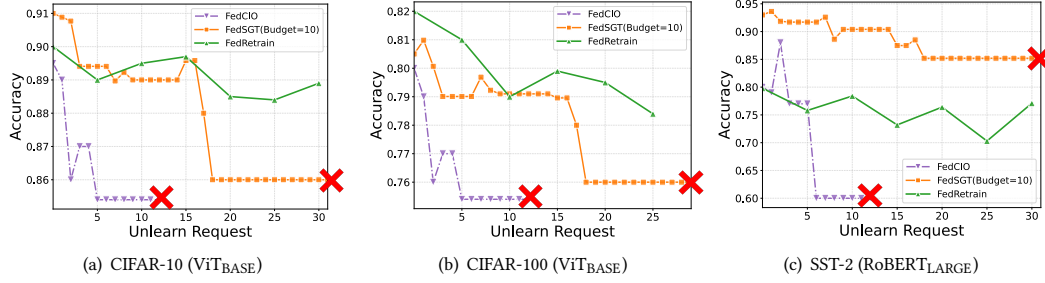


Figure 7: Unlearning Performance on Non-IID setting.

Table 3: Training time (hours). FedSGT uses $B = 10$ sequences; parentheses show per-sequence cost.

Dataset	Model	FedAvg	FedCIO	FedSGT
CIFAR-10	ViT _{BASE}	68.42	30.53	41.31 (4.13)
CIFAR-100	ViT _{BASE}	69.21	31.40	42.30 (4.20)
SST-2	RoBERT _{LARGE}	53.40	20.30	46.20 (4.62)
QQP	RoBERT _{LARGE}	182.44	112.35	181.0 (18.10)
QNLI	RoBERT _{LARGE}	75.25	63.41	70.30 (7.03)
MNLI	RoBERT _{LARGE}	252.51	182.36	241.3 (24.13)

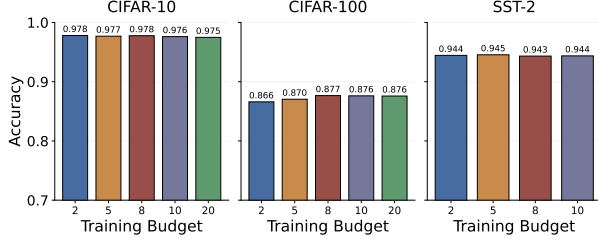


Figure 8: Training Results with different training budgets.

FedSGT is comparable to FedAvg and FedCIO, despite the larger absolute running time caused by the substantially heavier backbone models (e.g., ViT_{BASE} and RoBERT_{LARGE}). This empirical observation is consistent with our complexity analysis in Section 5.3: although FedSGT trains $B=10$ sequences, each sequence updates only a single lightweight PEFT module, and thus the per-sequence cost remains small (as reflected by the numbers in parentheses).

The deviations from the theoretical ratios primarily arise from practical system factors in our standalone FL simulation environment, including client-side parallelism effects, heterogeneous hardware throughput, and occasional communication stalls. In particular, FedAvg and FedCIO benefit from fully parallel client updates within each round, while FedSGT’s sequence-wise training introduces minor scheduling overhead. Nonetheless, the overall wall-clock time remains in the same order as FedAvg and FedCIO across all datasets, confirming that FedSGT achieves improved unlearning capabilities without incurring significant additional training cost.

6.5 Ablation Study

6.5.1 Training Budget Analysis. Figure 8 reports the training accuracy of FedSGT under different training budgets. Overall, the performance remains stable across a wide range of budgets. This behavior is expected: although larger budgets introduce more training sequences, each sequence corresponds to a different permutation of groups, and some permutations may lead to slightly stronger or weaker convergence. As a result, increasing the budget does not necessarily yield monotonic improvement. Instead, the accuracy variations stay within a small margin across all datasets, indicating that FedSGT is inherently robust to the choice of training budget. This also reflects the efficiency of our group-based training method: most useful information is already captured within a small number of sequences.

Unlearning Performance. Figure 9 illustrates the unlearning behavior of FedSGT under different training budgets. When the budget is smaller than the number of groups ($B < 10$), the system exhausts available sequences more quickly, leading to an earlier service termination. As the budget increases, FedSGT consistently supports a larger number of deletion requests, since more sequences remain valid after each unlearning event.

When the budget exceeds the group count ($B \geq 10$), the service maintenance time saturates and no longer improves with additional sequences. This is expected: beyond L sequences, many permutations become redundant, and the marginal gain in surviving deletions diminishes.

In terms of model utility during the unlearning process, smaller budgets may occasionally exhibit slightly better accuracy in the early stages due to randomness in sequence selection. However, generally larger budgets yield more stable performance and significantly longer service availability.

Takeaway 3: Increasing the training budget yields limited gains in training accuracy, but substantially improves unlearning robustness—larger budgets allow FedSGT to sustain significantly longer service availability under repeated deletion requests.

6.5.2 Client Slicing Number. Figure 10 and Figure 11 illustrate the impact of different client slice numbers on the training performance of FedSGT. Under the IID setting (Figure 10), varying the number of slices has only a minor effect on accuracy. Since client data are already well-balanced across the population, additional slicing does not substantially alter the group distributions and thus leads to comparable performance across all slice counts.

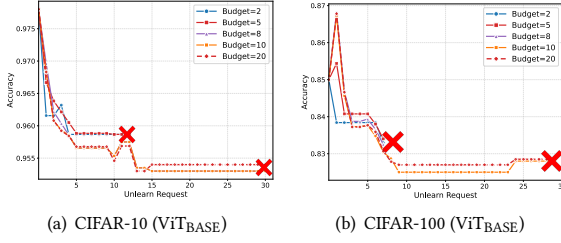


Figure 9: FedSGT’s unlearning performance with different training budget constraints on IID setting.

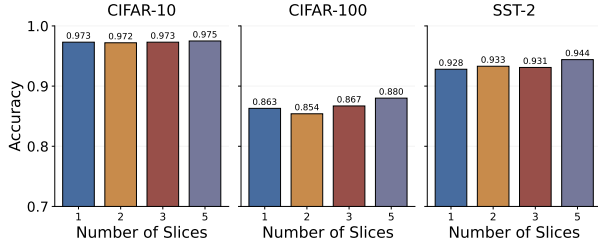


Figure 10: Training Performance with various client slice numbers under IID setting

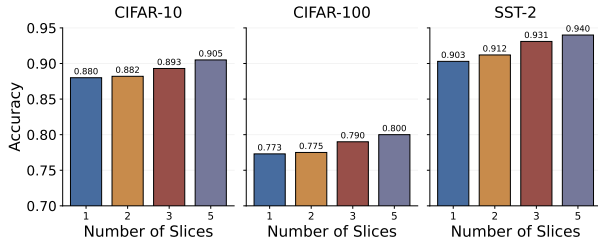


Figure 11: Training Performance with various client slice numbers under Non-IID setting

In contrast, under the Non-IID setting (Figure 11), increasing the slice number leads to noticeably higher accuracy. This improvement arises because finer slicing enables the uniform grouping strategy (Section 4.1) to redistribute heterogeneous client data more evenly across groups. As a result, each training phase encounters a more representative mixture of data, mitigating the adverse effects of data heterogeneity and improving overall model convergence.

Takeaway 4: *Client slicing matters more under Non-IID data. Under IID, different slice numbers yield similar performance. Under Non-IID, more slices substantially improve accuracy by balancing group composition. The tradeoff is higher communication participation as clients join more groups.*

6.5.3 Sequence Selection Method. We compare three sequence selection strategies—*LongSeq*, *MinSeq*, and *AllSeq*—to understand how the choice of active sequence affects unlearning performance. As

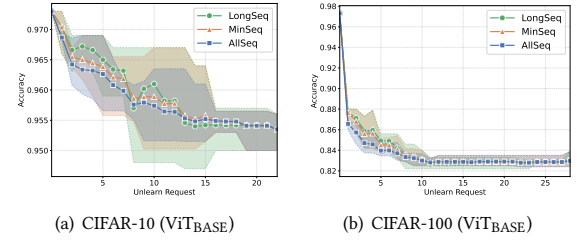


Figure 12: FedSGT’s unlearning performance with different sequence selection strategies.

shown in Figure 12, all three strategies yield similar average performance across unlearning requests, indicating that the sequence-aware design of FedSGT is generally robust to the selection rules.

However, the variability among strategies differs noticeably. *LongSeq*, which always selects the longest remaining sequence, exhibits the largest variance: depending on how many valid modules remain in that sequence, it may perform unusually well or degrade more rapidly. In contrast, *MinSeq* and *AllSeq* provide more stable behavior, as they rely on either choosing the shortest surviving sequence or averaging over all valid sequences, both of which reduce variance during deletion progression. We include further discussion in Appendix F.

Takeaway 5: *All sequence-selection strategies achieve similar average unlearning performance. LongSeq shows higher variance because relying on a single sequence can lead to either very strong or very weak outcomes, whereas MinSeq and AllSeq offer more stability.*

7 CONCLUSION

We presented FedSGT, an exact federated unlearning framework that partitions client data into balanced groups and maps each group to lightweight PEFT modules across multiple training sequences. By isolating data groups at the module level, FedSGT enables instant exact unlearning—deactivating modules tied to deletion requests without retraining—while keeping client storage and communication bounded. Theoretically, we analyze deletion rate, performance under accumulating deletions, and overhead; empirically, FedSGT matches or exceeds baselines in accuracy and training efficiency and sustains longer service cycles under continual unlearning. Extensive ablations confirm robustness to data heterogeneity and hyper-parameters.

REFERENCES

- [1] Gunnar Blom, Lars Holst, and Dennis Sandell. 2004. 7.5 coupon collecting i, 7.6 coupon collecting ii, and 15.4 coupon collecting iii. Problems and Snapshots from the World of Probability.
- [2] Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. Machine unlearning. In *2021 IEEE symposium on security and privacy (SP)*. IEEE, 141–159.
- [3] Xiaoyu Cao, Jinyuan Jia, Zaixi Zhang, and Neil Zhenqiang Gong. 2023. Fedrecovery: Recovering from poisoning attacks in federated learning using historical information. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1366–1383.
- [4] Chong Chen, Fei Sun, Min Zhang, and Bolin Ding. 2022. Recommendation unlearning. In *Proceedings of the ACM web conference 2022*. 2768–2777.

- [5] Min Chen, Zhikun Zhang, Tianhao Wang, Michael Backes, Mathias Humbert, and Yang Zhang. 2021. When machine unlearning jeopardizes privacy. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*. 896–911.
- [6] Somnath Basu Roy Chowdhury, Krzysztof Choromanski, Arijit Sehanobish, Avinava Dubey, and Snigdha Chaturvedi. 2024. Towards scalable exact machine unlearning using parameter-efficient fine-tuning. *arXiv preprint arXiv:2406.16257* (2024).
- [7] Alexey Dosovitskiy. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [8] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and trends® in theoretical computer science* 9, 3–4 (2014), 211–407.
- [9] Dawei Gao, Daoyuan Chen, Zitao Li, Yuexiang Xie, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. 2023. FS-real: A real-world cross-device federated learning platform. *Proceedings of the VLDB Endowment* 16, 12 (2023), 4046–4049.
- [10] Ronald L Graham, Donald E Knuth, and Oren Patashnik. 1994. *Concrete Mathematics: A Foundation for Computer Science*. Addison-Wesley.
- [11] Varun Gupta, Christopher Jung, Seth Neel, Aaron Roth, Saeed Sharif-Malvajerdi, and Chris Waites. 2021. Adaptive machine unlearning. *Advances in Neural Information Processing Systems* 34 (2021), 16319–16330.
- [12] Anisa Halimi, Swanand Kadhe, Amrith Rawat, and Nathalie Baracaldo. 2022. Federated unlearning: How to efficiently erase a client in fl? *arXiv preprint arXiv:2207.05521* (2022).
- [13] Mengde Han, Tianqing Zhu, Lefeng Zhang, Huan Huo, and Wanlei Zhou. 2025. Vertical federated unlearning via backdoor certification. *IEEE Transactions on Services Computing* (2025).
- [14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [15] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. 2021. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning* 14, 1–2 (2021), 1–210.
- [16] Richard M Karp. 2009. Reducibility among combinatorial problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 219–241.
- [17] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).
- [18] Weirui Kuang, Bingchen Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5260–5271.
- [19] Meghdad Kurmanji, Eleni Triantafillou, and Peter Triantafillou. 2024. Machine unlearning in learned databases: An experimental analysis. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26.
- [20] Hong kyu Lee, Qiuchen Zhang, Carl Yang, Jian Lou, Li Xiong, et al. 2025. Contrastive Unlearning: A Contrastive Approach to Machine Unlearning. In *34th International Joint Conference on Artificial Intelligence (IJCAI)*.
- [21] Guanghao Li, Li Shen, Jian Sun, Yue Hu, Han Hu, and Dacheng Tao. 2023. Subspace based federated unlearning. *arXiv preprint arXiv:2302.12448* (2023).
- [22] Qibin Li, Yiqun Diao, Quan Chen, and Bingsheng He. 2022. Federated learning on non-iid data silos: An experimental study. In *2022 IEEE 38th international conference on data engineering (ICDE)*. IEEE, 965–978.
- [23] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. 2020. Federated learning: Challenges, methods, and future directions. *IEEE signal processing magazine* 37, 3 (2020), 50–60.
- [24] Shen Lin, Xiaoyu Zhang, Chenyang Chen, Xiaofeng Chen, and Willy Susilo. 2023. Erm-kt: Knowledge-level machine unlearning via knowledge transfer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 20147–20155.
- [25] Gaoyang Liu, Xiaoqiang Ma, Yang Yang, Chen Wang, and Jiangchuan Liu. 2021. Federaser: Enabling efficient client-level data removal from federated learning models. In *2021 IEEE/ACM 29th international symposium on quality of service*. IEEE, 1–10.
- [26] Gaoyang Liu, Xiaoqiang Ma, Yang Yang, Chen Wang, and Jiangchuan Liu. 2021. Federaser: Enabling efficient client-level data removal from federated learning models. In *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*. IEEE, 1–10.
- [27] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin A Raffel. 2022. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *Advances in Neural Information Processing Systems* 35 (2022), 1950–1965.
- [28] Junxu Liu, Jian Lou, Li Xiong, Jingfei Liu, and Xiaofeng Meng. 2024. Cross-silo federated learning with record-level personalized differential privacy. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 303–317.
- [29] Ruixuan Liu, Yang Cao, Masatoshi Yoshikawa, and Hong Chen. 2020. FedSel: Federated sgd under local differential privacy with top-k dimension selection. In *International Conference on Database Systems for Advanced Applications*. Springer, 485–501.
- [30] Yang Liu, Yan Kang, Tianyuan Zou, Yanhong Pu, Yuanqin He, Xiaozhou Ye, Ye Ouyang, Ya-Qin Zhang, and Qiang Yang. 2024. Vertical federated learning: Concepts, advances, and challenges. *IEEE transactions on knowledge and data engineering* 36, 7 (2024), 3615–3634.
- [31] Yinhan Liu, Mylène Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [32] Ziyao Liu, Yu Jiang, Jiyuan Shen, Minyi Peng, Kwok-Yan Lam, and Xingliang Yuan. 2023. A survey on federated unlearning: Challenges, methods, and future directions. *arXiv preprint arXiv:2310.20448* (2023).
- [33] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*. PMLR, 1273–1282.
- [34] Thanh Tam Nguyen, Thanh Trung Huynh, Zhao Ren, Phi Le Nguyen, Alan Wee-Chung Liew, Hongzhi Yin, and Quoc Viet Hung Nguyen. 2025. A survey of machine unlearning. *ACM Transactions on Intelligent Systems and Technology* 16, 5 (2025), 1–46.
- [35] Zibin Pan, Zhichao Wang, Chi Li, Kaiyan Zheng, Boqi Wang, Xiaoying Tang, and Junhua Zhao. 2025. Federated unlearning with gradient descent and conflict mitigation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 19804–19812.
- [36] Hongyu Qiu, Yongwei Wang, Yonghui Xu, Lizhen Cui, and Zhiqi Shen. 2023. Fedcio: Efficient exact federated unlearning with clustering, isolation, and one-shot aggregation. In *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 5559–5568.
- [37] Gobinda Saha, Isha Garg, and Kaushik Roy. 2021. Gradient projection memory for continual learning. *arXiv preprint arXiv:2103.09762* (2021).
- [38] Ramesh Sitaraman. 2001. The power of two random choices: A survey of techniques and results. (2001).
- [39] Youming Tao, Cheng-Long Wang, Miao Pan, Dongxiao Yu, Xiuzhen Cheng, and Di Wang. 2024. Communication Efficient and Provable Federated Unlearning. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1119–1131.
- [40] Jana Vatter, Ruben Mayer, and Hans-Arno Jacobsen. 2023. The evolution of distributed systems for graph neural networks and their origin in graph processing and deep learning: A survey. *Comput. Surveys* 56, 1 (2023), 1–37.
- [41] Paul Voigt and Axel Von dem Bussche. 2017. The eu general data protection regulation (gdpr). *A practical guide, 1st ed.*, Cham: Springer International Publishing 10, 3152676 (2017), 10–5555.
- [42] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461* (2018).
- [43] Fei Wang, Baochun Li, and Bo Li. 2023. Federated unlearning and its privacy threats. *IEEE Network* 38, 2 (2023), 294–300.
- [44] Zichen Wang, Xiangshan Gao, Cong Wang, Peng Cheng, and Jiming Chen. 2024. Efficient vertical federated unlearning via fast retraining. *ACM Transactions on Internet Technology* 24, 2 (2024), 1–22.
- [45] Kang Wei, Jun Li, Ming Ding, Chuan Ma, Howard H Yang, Farhad Farokhi, Shi Jin, Tony QS Quek, and H Vincent Poor. 2020. Federated learning with differential privacy: Algorithms and performance analysis. *IEEE transactions on information forensics and security* 15 (2020), 3454–3469.
- [46] Leijie Wu, Song Guo, Junxiao Wang, Zicong Hong, Jie Zhang, and Yaohong Ding. 2022. Federated unlearning: Guarantee the right of clients to forget. *IEEE Network* 36, 5 (2022), 129–135.
- [47] Leijie Wu, Song Guo, Junxiao Wang, Zicong Hong, Jie Zhang, and Yaohong Ding. 2022. Federated unlearning: Guarantee the right of clients to forget. *IEEE Network* 36, 5 (2022), 129–135.
- [48] Haocheng Xia, Jinfei Liu, Jian Lou, Zhan Qin, Kui Ren, Yang Cao, and Li Xiong. 2023. Equitable data valuation meets the right to be forgotten in model markets. *Proceedings of the VLDB Endowment* 16, 11 (2023).
- [49] Xiaoyu Xia, Ziqi Wang, Ruoxi Sun, Bowen Liu, Ibrahim Khalil, and Minhui Xue. 2025. Edge Unlearning is Not “on Edge”: An Adaptive Exact Unlearning System on Resource-Constrained Devices. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2546–2563.
- [50] Zuobin Xiong, Wei Li, Yingshu Li, and Zhipeng Cai. 2023. Exact-fun: an exact and efficient federated unlearning approach. In *2023 IEEE International Conference on Data Mining (ICDM)*. IEEE, 1439–1444.
- [51] Haonan Yan, Xiaoguang Li, Ziyao Guo, Hui Li, Fenghua Li, and Xiaodong Lin. 2022. ARCANE: An Efficient Architecture for Exact Machine Unlearning. In *IJCAI*, Vol. 6. 19.
- [52] Guangsheng Yu, Yanna Jiang, Qin Wang, Xu Wang, Baihe Ma, Caijun Sun, Wei Ni, and Ren Ping Liu. 2023. Split unlearning. *arXiv preprint arXiv:2308.10422* (2023).

- [53] Bokang Zhang, Yanglin Zhang, Zhikun Zhang, Jinglan Yang, Lingying Huang, and Junfeng Wu. 2024. S2NeRF: Privacy-preserving Training Framework for NeRF. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 258–272.
- [54] Chen Zhang, Yu Xie, Hang Bai, Bin Yu, Weihong Li, and Yuan Gao. 2021. A survey on federated learning. *Knowledge-Based Systems* 216 (2021), 106775.
- [55] Zhuo Zhang, Yuanhang Yang, Yong Dai, Lizhen Qu, and Zenglin Xu. 2022. When Federated Learning Meets Pre-trained Language Models' Parameter-Efficient Tuning Methods. *arXiv preprint arXiv:2212.10025* (2022).
- [56] Ligeng Zhu, Zhijian Liu, and Song Han. 2019. Deep leakage from gradients. *Advances in neural information processing systems* 32 (2019).

APPENDIX

A DETAILED THEORITICAL ANALYSIS

A.1 Detailed Deletion Rate Derivation

PROOF. Although FedSGT differs from S3T in its training method, it adopts a similar sharding and slicing configuration. In FedSGT, however, the number of *shards* is fixed at 1, and multiple slices from different clients jointly form a *group*, which serves the same functional role as a slice in S3T. Each unlearning request randomly affects one cluster, following a similar mechanism as in S3T.

The deletion rate is the expected number of requests to delete all the topmost $B' = \min(L, B)$ slices. Using linearity of expectation, the deletion rate is

$$\delta(\text{FedSGT}) = \mathbb{E}[t_1 + t_2 + \dots + t_{B'}] \quad (5)$$

$$= \mathbb{E}[t_1] + \mathbb{E}[t_2] + \dots + \mathbb{E}[t_{B'}] \quad (6)$$

where $E[t_i] = 1/p_i$. The denominator p_i is the probability that the i -th slice is affected after $(i - 1)$ slices are affected, which is given by

$$p_i \approx \frac{B' - (i - 1)}{L}$$

The approximation in the above equation is based on the assumption that the number of unlearning requests is small enough that the group sizes remain unchanged. Substituting the above result into Equation (6), we have:

$$\begin{aligned} \delta(\text{FedSGT}) &= \frac{L}{B' - 0} + \frac{L}{B' - 1} + \dots + \frac{L}{1} \\ &= L \left(\frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{B'} \right) \\ &\equiv L \cdot H_{B'} \end{aligned}$$

Similarly, deletion rate for FedCIO is the expected number of requests to affect all the c clusters:

$$\delta(\text{FedCIO}) = \mathbb{E}[t_1 + t_2 + \dots + t_c] \quad (7)$$

$$= \mathbb{E}[t_1] + \mathbb{E}[t_2] + \dots + \mathbb{E}[t_c] \quad (8)$$

where $E[t_i] = 1/p_i$. Since there are totally c clusters, the expression p_i is given by:

$$p_i \approx \frac{c - (i - 1)}{c}$$

Again, we assume that the number of unlearning requests is much smaller than the cluster size. Substituting the expression of p_i into

Equation (8), we get:

$$\begin{aligned} \delta(\text{FedCIO}) &= \frac{c}{c - 0} + \frac{c}{c - 1} + \dots + \frac{c}{1} \\ &= c \left(\frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{c} \right) \\ &\equiv c \cdot H_c \end{aligned}$$

□

A.2 Unlearning Performance Analysis

PROOF. To simulate the unlearning process, let variable X_i denote the cluster id that the i th unlearning request affects, which is uniformly drawn from the integers from 0 to $L - 1$, where i is from 1 to r . Following the cyclic rotation algorithm in S3T, we want to find the number of remaining training samples in the sequence with the largest remaining clusters. Therefore, we define *cyclic span* (U) as the minimum number of slices to be removed in a sequence using the cyclic rotation algorithm with $B \geq L$. It is mathematically written as $U = \min_j \{ \max_i [(X_i + j) \% L] - \min_i [(X_i + j) \% L] \}$ where j is an integer from 0 to $L - 1$. Now, we want to find the expectation of T because the number of the largest remaining training samples is

$$R = \frac{|D|}{L} \cdot (L - T) \quad (9)$$

A cleaner, equivalent way to write T (and easier to reason about) is: Let $Z = z_1 < z_2 < \dots < z_m$ be the distinct sampled values among $(X_1, \dots, X_r)$. Define the cyclic gaps

$$d_i = \begin{cases} z_{i+1} - z_i, & i = 1, \dots, m - 1 \\ z_1 + L - z_m, & i = m \end{cases}$$

Then $T = L - \max_{1 \leq i \leq m} d_i + 1$. The intuition is as follows: Rotating by j is the same as choosing where to “cut” the circle into a line segment $([0, L))$. The best cut is across the largest empty arc between consecutive occupied points; the span that needs to cover the points is the complement of that gap, i.e., $(L - \max d_i) + 1$.

Let M be the number of distinct occupied positions (so $1 \leq M \leq \min(r, L)$). Then

$$\Pr(M = m) = \binom{L}{m} \cdot m! \cdot S(r, m) / L^r \quad (10)$$

, where $S(r, m)$ is the Stirling number of the second kind. Conditional on $M = m$, the ordered gaps $(d_1, \dots, d_m)$ are uniformly distributed over all m -tuples of positive integers summing to L . Hence, for any $s \geq 1$,

$$\begin{aligned} \Pr \left(\max_i d_i \leq s \mid M = m \right) &= \frac{1}{\binom{L-1}{m-1}} \times \sum_{j=0}^{\lfloor \frac{L-m}{s} \rfloor} (-1)^j \binom{m}{j} \binom{L-1-j}{m-1} \end{aligned} \quad (11)$$

From this,

$$\mathbb{E}[U \mid M = m] = L - \mathbb{E}[\max_i d_i \mid M = m] + 1 \quad (12)$$

$$= 1 + L - \sum_{t=1}^L \Pr(\max_i d_i \geq t \mid M = m) \quad (13)$$

$$= 1 + \sum_{s=1}^{L-1} \Pr(\max_i d_i \leq s \mid M = m) \quad (14)$$

and finally, by the definition of expectation,

$$\mathbb{E}[U] = \sum_{m=1}^{\min(r,L)} \Pr(M = m) \cdot \mathbb{E}[U \mid M = m]. \quad (15)$$

Applying linearity of expectation to Equation (9), we have $\mathbb{E}[R] = \frac{|D|}{L} (L - \mathbb{E}[U])$. $\square$

To clarify the above equations, we provide a detailed explanation below. To interpret Equation (10), consider the r draws as placing r labeled balls into L labeled bins indexed by $(0, \dots, L-1)$. Since each draw is equally probable, there are L^r possible assignments. The event $M = m$ corresponds to the case in which exactly m bins are nonempty. To compute the probability of this event, we first select which m bins are occupied, which can be done in $\binom{L}{m}$ ways. We then distribute the r labeled balls among these m bins such that none remains empty. The number of surjective mappings from an r -element labeled set to an m -element labeled set is $m!S(r, m)$, where $S(r, m)$ denotes the Stirling number of the second kind. Specifically, $S(r, m)$ represents the number of ways to partition the r labeled elements into m nonempty unlabeled subsets, while the multiplicative factor $m!$ accounts for assigning these subsets to the m labeled bins.

The denominator of Equation (11) is the total number of possible positive m -tuples that sum up to L . It is equivalent to select $m-1$ bars out of $L-1$ possible positions to divide L objects into m partitions. Next, we count those with $\max d_i \leq s$. Write $z_i = d_i - 1$. Then $z_i \geq 0$, $\sum z_i = L - m$, and $\max d_i \leq s \iff z_i \leq s - 1$ for all i . Let (A_i) be the event $(z_i \geq s)$. For a fixed set (J) of size (j) , substitute $(z'_i = z_i - s)$ for $(i \in J)$. Then

$$\sum_{i \notin J} z_i + \sum_{i \in J} z'_i = (L - m) - js$$

which has $\binom{(L-m)-js+m-1}{m-1} = \binom{L-1-js}{m-1}$ solutions if $(L-m) - js \geq 0$, thus j take values from 0 to $\lfloor (L-m)/s \rfloor$. By the inclusion-exclusion principle,

$$\#\{\max d_i \leq s\} = \sum_{j=0}^{\lfloor (L-m)/s \rfloor} (-1)^j \binom{m}{j} \binom{L-1-js}{m-1}.$$

Equation (12) comes from applying linearity of expectation to its definition. From Equation (12) to Equation (13), we leverage the fact that d_i is a non-negative integer-valued random variable and applied the tail-sum formula $\mathbb{E}[X] = \sum_{k=1}^{\infty} \Pr(X \geq k)$, where X is a non-negative random integer. To see why from Equation (13) to Equation (14), let $F_m(s) \equiv \Pr(\max_i d_i \leq s \mid M = m)$, whose expression is Equation (11) for $s \geq 1$ and 0 for $s = 0$, i.e., $F_m(0) = 0$.

Then

$$\begin{aligned} \Pr(\max_i d_i \geq t \mid M = m) &= 1 - \Pr(\max_i d_i \leq t-1 \mid M = m) \\ &= 1 - F_m(t-1) \end{aligned}$$

Substitute it in to Equation (13),

$$\begin{aligned} \mathbb{E}[U \mid M = m] &= 1 + L - \sum_{t=1}^L \Pr(\max_i d_i \geq t \mid M = m) \\ &= 1 + L - \sum_{t=1}^L [1 - F_m(t-1)] \\ &= 1 + \sum_{t=1}^L F_m(t-1) \end{aligned}$$

Renaming $s = t - 1$ gives a clean form

$$\mathbb{E}[U \mid M = m] = 1 + \sum_{s=0}^{L-1} F_m(s) = 1 + \sum_{s=1}^{L-1} F_m(s)$$

, where the last equality is because $F_m(0) = 0$ as defined earlier. By substituting the definition of $F_m(s)$, we get Equation (14).

A.3 Detailed Communication Cost Derivation

In this section, we derive the full expression of the expected communication cost for a client under uniform random grouping. Consider a client partitioned into S slices, and recall that all NS slices are uniformly assigned to L groups following Algorithm 1. Let K denote the number of distinct groups that contain at least one slice of this client. The total communication cost is determined by K .

(1) *Communication cost conditioned on $K = k$.* Suppose the client belongs to exactly k groups. Across the L cyclic training sequences, let $X_1, \dots, X_k$ denote the positions of these groups in a given sequence, and let $V = \min_i X_i$. Once the first such group appears (position V), the client participates in all subsequent rounds, yielding a per-sequence cost of $L - V + 1$. Since the k positions form a uniform subset of $\{1, \dots, L\}$, the minimum order statistic satisfies $\mathbb{E}[V] = (L+1)/(k+1)$, and therefore

$$\mathbb{E}[C_{\text{seq}} \mid K = k] = (L+1) \frac{k}{k+1}.$$

Since FedSGT executes all L cyclic sequences,

$$\mathbb{E}[C_{\text{total}}(L) \mid K = k] = L(L+1) \frac{k}{k+1}. \quad (16)$$

(2) *Distribution of K under random grouping.* Each of the S labeled slices independently selects one of the L groups, yielding L^S possible assignments. To obtain exactly k occupied groups, we first choose which k groups are used ($\binom{L}{k}$ choices). Then we assign the S slices onto these k labeled groups such that each group receives at least one slice. The number of such onto assignments is $k! \{S\}_k$, where $\{S\}_k$ is the Stirling number [10] of the second kind. Therefore,

$$\Pr(K = k) = \frac{\binom{L}{k} k! \{S\}_k}{L^S}, \quad k = 1, \dots, \min\{S, L\}. \quad (17)$$

(3) *Final closed-form expectation.* Combining (16) and (17), the unconditional expected communication cost is

$$\mathbb{E}[C_{\text{total}}(L, S)] = \sum_{k=1}^{\min\{S, L\}} \Pr(K = k) L(L+1) \frac{k}{k+1} \quad (18)$$

$$= L(L+1) \sum_{k=1}^{\min\{S, L\}} \frac{\binom{L}{k} k! \frac{S!}{k!}}{L^S} \frac{k}{k+1}. \quad (19)$$

Equation (19) gives the exact expected communication overhead of a client as a function of S , the slicing granularity of FedSGT.

B PERFORMANCE-OVERHEAD TRADE-OFF UNDER HETEROGENEOUS FL DEVICES

In federated settings, edge devices can be highly heterogeneous in their computation and communication capabilities. To better accommodate this heterogeneity, FedSGT allows using different slicing granularities S for different clients. Intuitively, a larger S provides finer-grained data partitioning and typically improves model performance (as observed in subsection 6.5.2), but it also increases the client-side communication overhead. Hence, S serves as a practical parameter to balance model quality and device affordability.

Communication overhead increases with S . The slicing granularity S affects the client communication overhead through the number of occupied groups K . As S increases, the client's slices are more likely to spread across more groups, which increases the expected number of occupied groups:

$$\mathbb{E}[K] = L \left(1 - \left(1 - \frac{1}{L} \right)^S \right),$$

which is monotone increasing in S . Meanwhile, conditioned on $K = k$, the expected communication cost is

$$\mathbb{E}[C_{\text{total}}(L) \mid K = k] = L(L+1) \frac{k}{k+1},$$

which is strictly increasing in k . Therefore, the expected communication cost is monotonically non-decreasing in S , increasing from

$$\mathbb{E}[C_{\text{total}}(L, 1)] = \frac{L(L+1)}{2}$$

toward the saturation limit

$$\lim_{S \rightarrow \infty} \mathbb{E}[C_{\text{total}}(L, S)] = L^2.$$

Training cost is relatively insensitive to S . Although a larger S makes a client appear in more groups (thus increasing communication/activation rounds), it does not necessarily increase local training computation. The reason is that the client's total local data size remains fixed; slicing only changes how the same data are distributed across groups. Under uniform random grouping and cyclic ordering, each slice has the same expected reuse count across phases, so the expected training cost mainly depends on the total local data size, rather than the slicing granularity S .

Practical implication. This property enables a flexible deployment strategy for heterogeneous edge devices: clients with stronger communication budgets can use a larger S to obtain better model performance, while communication-constrained devices can use

a smaller S to reduce communication overhead, with only limited impact on local training cost.

C ABLATION STUDY ABOUT OTHER PEFT

In addition to LoRA, we consider two representative PEFT strategies that are widely used in NLP models: *Prefix Tuning* and *adapters*. **Prefix Tuning** optimizes a small set of continuous *virtual tokens* (prefixes) that are prepended to the key/value states (or input embeddings) of each Transformer layer, while keeping the backbone frozen. This approach steers model behavior by modifying the attention context with a compact number of trainable parameters controlled by the prefix length. **Adapters** generally insert lightweight task-specific modules into Transformer blocks while keeping the backbone frozen. In our implementation, we instantiate this adapter-style PEFT baseline using **IA3** (Infused Adapter by Inhibiting and Amplifying Inner Activations [27]), which adapts the model by learning multiplicative scaling vectors on internal activations rather than inserting conventional bottleneck layers. For clarity and consistency, we refer to this baseline as IA3 in the remainder of this section.

Setup. To fairly compare different PEFT choices in FedSGT, we keep the federated training pipeline and data grouping identical across methods, and only change the PEFT-specific modules and their associated optimization hyperparameters. All experiments are conducted on SST-2 using the same backbone model (RoBERTa_{LARGE}).

We use Prefix Tuning with 20 virtual tokens and a prefix hidden size of 512. We optimize the prefix parameters with a learning rate of 5×10^{-3} , while freezing the backbone model parameters. IA3 introduces lightweight multiplicative scaling parameters on internal activations while keeping the backbone frozen. We optimize IA3 parameters with a learning rate of 1×10^{-3} .

Learning Performance. Figure 13 shows the learning curves of FedSGT with three PEFT choices (LoRA, Prefix Tuning, and IA3) under different sequence orders. Overall, all three methods achieve very similar final utility on SST-2, indicating that FedSGT is compatible with multiple PEFT backbones and does not rely on a specific adaptation module. In particular, LoRA achieves the best final accuracy (0.938), while Prefix Tuning and IA3 obtain 0.935 and 0.932, respectively. Although LoRA performs slightly better at performance, the margin is small, suggesting comparable end-task performance across the three PEFT strategies.

In terms of training dynamics, Prefix Tuning and IA3 exhibit more stable trajectories across rounds (and across different orders), with smoother improvement and smaller fluctuation during early-to-mid training stages. By contrast, LoRA shows larger variance in the training process, especially in the earlier rounds, but catches up quickly and eventually converges to the best final accuracy. These results suggest a trade-off between *training stability* and *peak performance*: Prefix Tuning and IA3 provide more stable optimization behavior, whereas LoRA offers a slightly stronger final performance in our setting.

Unlearning Performance. Figure 14 compares the unlearning-time utility of FedSGT with different PEFT choices under a sequence of unlearning requests. Overall, the three curves exhibit highly similar trend patterns. This is expected because all PEFT variants follow

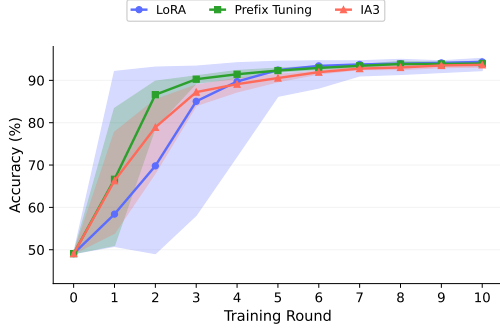


Figure 13: Learning curve of different PEFT in FedSGT



Figure 14: Unlearning curve of different PEFT in FedSGT

the same unlearning pipeline in FedSGT: for each unlearning request, they select available checkpoints using the same sequence selection method as described in Section 4.3. As a result, the major turning points and stage-wise transitions are aligned across methods.

In terms of utility during unlearning, the three PEFT methods remain close throughout the process, and their performance may alternately surpass one another under different requests. This indicates that FedSGT is broadly compatible with different PEFT methods in the unlearning stage, and no single PEFT choice dominates at every request step. Meanwhile, in the later stage (when only one group of data remains available), Prefix Tuning and IA3 achieve slightly better utility than LoRA. The reason is that Prefix Tuning and IA3 provide better adaptation capacity in this low-data regime, leading to stronger service utility after repeated unlearning updates.

Efficiency. Table 4 reports the training time of FedSGT on SST-2 with different PEFT choices. Overall, the three methods have comparable training cost under the same federated pipeline, indicating that replacing the PEFT module does not substantially change the end-to-end runtime of FedSGT. Among them, LoRA is the fastest (4.62 hours per sequence), IA3 (Adapters) is slightly slower (5.38 hours per sequence), and Prefix Tuning takes the longest time (5.57 hours per sequence).

Although IA3 has a smaller trainable parameter set and may appear computationally lighter in principle, the observed wall-clock time is affected by more than parameter count alone. In our federated implementation, the end-to-end runtime also includes

Table 4: Training time (hours) of FedSGT on SST-2. FedSGT uses $B = 10$ sequences; parentheses show per-sequence cost.

Dataset	LoRA	Prefix Tuning	Adapters
SST-2	46.20 (4.62)	55.74 (5.57)	53.79 (5.38)

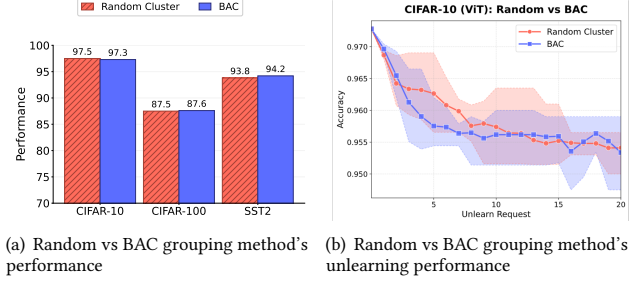


Figure 15: FedSGT's unlearning performance with different training budget constraints.

framework-level overheads (e.g., PEFT module wrapping, optimizer/kernel efficiency, communication and synchronization, checkpoint saving, and evaluation), and these overheads can dominate or offset the theoretical savings of a lighter PEFT parameterization. Similarly, Prefix Tuning introduces additional prefix-related operations in attention, which can increase practical runtime despite its parameter efficiency. Therefore, our results suggest that wall-clock efficiency is implementation- and system-dependent, and LoRA provides the best practical efficiency among the three PEFT choices while all methods remain in a similar efficiency range.

Summary. Overall, the results indicate that FedSGT is compatible with different PEFT choices, including LoRA, Prefix Tuning, and IA3. Across these PEFT methods, the differences in learning performance, unlearning performance, and efficiency are relatively small in our SST-2 experiments. This suggests that the effectiveness of FedSGT does not rely on a specific PEFT instantiation. Among them, LoRA shows a marginal advantage in final accuracy and runtime. LoRA may also be a favorable default choice in practice due to its broad adoption, good architectural compatibility, and mature tooling support.

D ABLATION STUDY ABOUT GROUPING METHOD

To study the impact of different grouping methods, we further design a **Barycenter-Aware Centering (BAC)** grouping method. Different from conventional clustering methods that mainly emphasize *intra-cluster similarity*, our goal here is different: we expect *each group* to preserve as much *global information* as possible. This design is particularly desirable for the unlearning stage, because as groups are progressively removed due to unlearning requests, the remaining groups can still maintain a relatively high utility if each group is individually representative of the overall data distribution.

The detailed procedure is shown in Algorithm 3. We first compute a slice-level embedding for each client slice using a pretrained

Algorithm 3: Barycenter-Aware Centering (BAC) Grouping Method

```

1 Input: Client set  $C = \{c_1, \dots, c_N\}$ ; pretrained encoder  $E(\cdot)$ ;
   per-client slice counts  $\{D_i\}_{i=1}^N$ ; number of base clusters  $K$ ;
   number of groups  $G$ 
2 Output: Groups  $\mathcal{G} = \{G_1, \dots, G_G\}$ 
3 Procedures:
   1: (Client-side slicing and embedding)
   2: for each client  $c_i \in C$  do
   3:   Partition local data into slices  $\{S_i^1, \dots, S_i^{D_i}\}$ 
   4:   for  $s = 1$  to  $D_i$  do
   5:      $e_i^s \leftarrow \frac{1}{|S_i^s|} \sum_{x \in S_i^s} E(x)$  // slice embedding
   6:   end for
   7:   Send  $\{e_i^1, \dots, e_i^{D_i}\}$  to server // no raw data sharing
   8: end for
   9: (Server-side equal-size semantic base clustering)
  10:  $E_{\text{all}} \leftarrow \{e_i^s \mid i \in [1..N], s \in [1..D_i]\}$ 
  11:  $M \leftarrow \sum_{i=1}^N D_i$  // total number of slices
  12:  $g \leftarrow \frac{1}{M} \sum_{i=1}^N \sum_{s=1}^{D_i} e_i^s$  // global centroid
  13: Run balanced clustering on  $E_{\text{all}}$  to obtain  $\mathcal{B} = \{B_1, \dots, B_K\}$ 
     with  $|B_l| = G$  for all  $l$  // equal-size base clusters
  14: (Barycenter matching across base clusters)
  15: for  $j = 1$  to  $G$  do
  16:    $T_j^{(0)} \leftarrow \mathbf{0}$ ;  $G_j \leftarrow \emptyset$  // initialize group sum and group set
  17: end for
  18: for  $l = 1$  to  $K$  do
  19:   Order items in  $B_l$  by a fixed deterministic key, and write
      $B_l = \{b_{l,1}, \dots, b_{l,G}\}$ 
  20:   for  $j = 1$  to  $G$  do
  21:     for  $u = 1$  to  $G$  do
  22:        $C_{j,u}^{(l)} \leftarrow \left\| T_j^{(l-1)} + e(b_{l,u}) - l g \right\|_2^2$  // stage- $l$  target sum
  23:     end for
  24:   end for
  25:    $\pi_l^* \leftarrow \arg \min_{\pi_l \in S_G} \sum_{j=1}^G C_{j,\pi_l(j)}^{(l)}$ 
  26:   Solve the above one-to-one assignment using the
     Hungarian algorithm with deterministic tie-breaking
  27:   for  $j = 1$  to  $G$  do
  28:      $G_j \leftarrow G_j \cup \{b_{l,\pi_l^*(j)}\}$ 
  29:      $T_j^{(l)} \leftarrow T_j^{(l-1)} + e(b_{l,\pi_l^*(j)})$ 
  30:   end for
  31: end for
Return: Group set  $\mathcal{G} = \{G_1, \dots, G_G\}$ 

```

encoder, and then aggregate all slice embeddings to obtain the *global centroid* (i.e., the barycenter of all slices). Instead of directly using standard clustering as the final grouping result, BAC first constructs *equal-size semantic base clusters* over all slice embeddings, and then performs a deterministic matching process across these base clusters to form the final groups. In particular, BAC assigns one slice from each base cluster to each group, while minimizing the deviation between each group’s cumulative embedding sum and the stage-wise target centered at the global centroid. As a result,

the centroid of each final group is encouraged to be close to the global centroid, making each group more globally representative.

Intuitively, BAC can be viewed as a *centroid-balanced grouping* strategy: it aims to preserve semantic diversity by drawing slices from different base clusters, while encouraging global representativeness through barycenter-aware matching. By design, this property is expected to be beneficial in our setting, where the model needs to sustain service quality under repeated unlearning requests and progressive loss of group-level data.

Effect of BAC Grouping. We compare the proposed BAC grouping method with random grouping in terms of both learning performance and unlearning performance. As shown in Figure 15(a), BAC and random grouping achieve very similar learning performance across datasets (CIFAR-10, CIFAR-100, and SST-2), with only marginal differences in final utility. This suggests that, under our current settings, replacing random grouping with BAC does not substantially change the standard training performance of FedSGT.

We further compare their behavior during unlearning in Figure 15(b). Overall, BAC and random grouping exhibit similar utility trajectories, and BAC does not show a consistently large advantage in this experiment. A possible explanation is that random grouping already produces groups that are sufficiently representative of the global data distribution in our setting (e.g., due to the slicing and shuffling procedure), which reduces the room for further improvement from barycenter-aware grouping.

From an efficiency perspective, BAC incurs additional preprocessing overhead in the grouping stage. In CIFAR-10 experiments, the grouping time of BAC is **102** seconds, compared with **8** seconds for random grouping. Therefore, although BAC is motivated by improving group-level representativeness, its empirical benefit is limited in our current experiments while introducing extra grouping cost.

Overall, these results suggest that random grouping is already sufficient in our current setting, offering a better trade-off between efficiency and effectiveness.

E RELATED WORK

Approximate Federated Unlearning. Many FU methods use approximate strategies that reduce a client’s influence without guaranteeing exact removal. A major line of work is gradient-ascent(GA)-based unlearning [32, 47], which increases the forgotten client’s loss, but direct GA often severely harms the utility of retained clients [12]. To address this, EWCSGA [47] adds regularization to limit utility drop. Another family computes update directions orthogonal to the input subspace [21, 37], which works in centralized settings but is unsuitable for FL due to privacy leakage from transmitting input features. More recent work [35] refines GA by stabilizing the loss and deriving update directions that emphasize the forgotten client while minimally conflicting with retained ones, reducing utility degradation.

Vertical Federated Unlearning. While our work focuses on horizontal FL, vertical FL has also gained attention, where different parties hold complementary feature spaces. Recent studies propose efficient and privacy-aware VFL training frameworks [30, 53], but unlearning in VFL [13, 44] is particularly challenging since deleting one user’s data requires coordinated removal across multiple parties.

Table 5: The hyperparameters used in experiments for FedSGT, FedCIO and FedAvg across different benchmarks and Transformer model variants.

Dataset	Model	Rank (r)	Alpha (α)	Learning Rate
<i>Vision Tasks</i>				
CIFAR-10	ViT _{BASE}	16	16	2e-3
CIFAR-100	ViT _{BASE}	16	16	2e-3
<i>Language Tasks (GLUE)</i>				
SST-2	RoBERT _{LARGE}	16	32	1e-4
QQP	RoBERT _{LARGE}	16	32	1e-4
QNLI	RoBERT _{LARGE}	16	32	1e-4
MNLI	RoBERT _{LARGE}	16	32	1e-4

Existing approaches such as Split-U [52] provide early solutions but still incur notable computational or communication cost.

Machine Unlearning Privacy Leakage. Recent work [5] shows that the unlearning process itself may introduce new privacy risks, as adversaries can potentially infer membership or sensitive attributes of the unlearned data from model updates. Such leakage risks naturally extend to the federated setting [43], where distributed updates and client-server interactions may further amplify the exposure surface. Although our work focuses on exact unlearning correctness and efficiency, mitigating privacy leakage during the unlearning process remains an important direction for future federated unlearning research.

Privacy-preserving FL. FL raises privacy concerns [28, 29, 45, 56], since exchanged updates can leak information about client data through gradient or activation inversion. While our work focuses on FU, we note that the training stage of most FU frameworks inherits the same privacy risks as standard FL. Improving training-time privacy (e.g., via differential privacy [8]) is largely orthogonal to our FU design and represents a promising future work.

F DISCUSSION

Efficient Retraining. Although our primary objective is to maintain uninterrupted service and minimize retraining whenever possible, certain applications may still require partial model recovery after several deletions to preserve performance. This approach is preferable to retraining the entire model from scratch once all data groups or clusters are affected, as otherwise the system may experience performance degradation due to a reduced training dataset size. FedCIO supports efficient retraining by updating only the clients within the affected cluster. Similarly, FedSGT can accommodate retraining through both online and offline processes. Specifically, FedSGT first identifies the sequence with the least number of affected groups and performs online retraining on those groups, excluding the unlearned data points. This minimizes the training cost while recovering model performance on the remaining dataset. In parallel, other sequences are retrained offline to proactively prepare for potential future unlearning requests.

Unlearning Request Distribution. In this paper, we assume that unlearning requests arrive randomly, meaning the deleted data samples are independent across requests. However, in certain real-world scenarios, unlearning requests may be related. For example,

in a healthcare FL system involving multiple hospitals, it is plausible that one hospital frequently issues consecutive unlearning requests, while others rarely do so.

Such unlearning patterns can be incorporated into FedSGT’s design. By adjusting the grouping strategy so that data from highly related clients are more likely to fall into the same group, FedSGT can reduce the number of affected groups when deletions occur. Concentrating related data into fewer groups allows the system to preserve more valid sequences after each unlearning request, thereby improving robustness and reducing degradation of model utility.

G EXPERIMENTAL SETUP

Dataset-specific hyperparameters are summarized in Table 5.